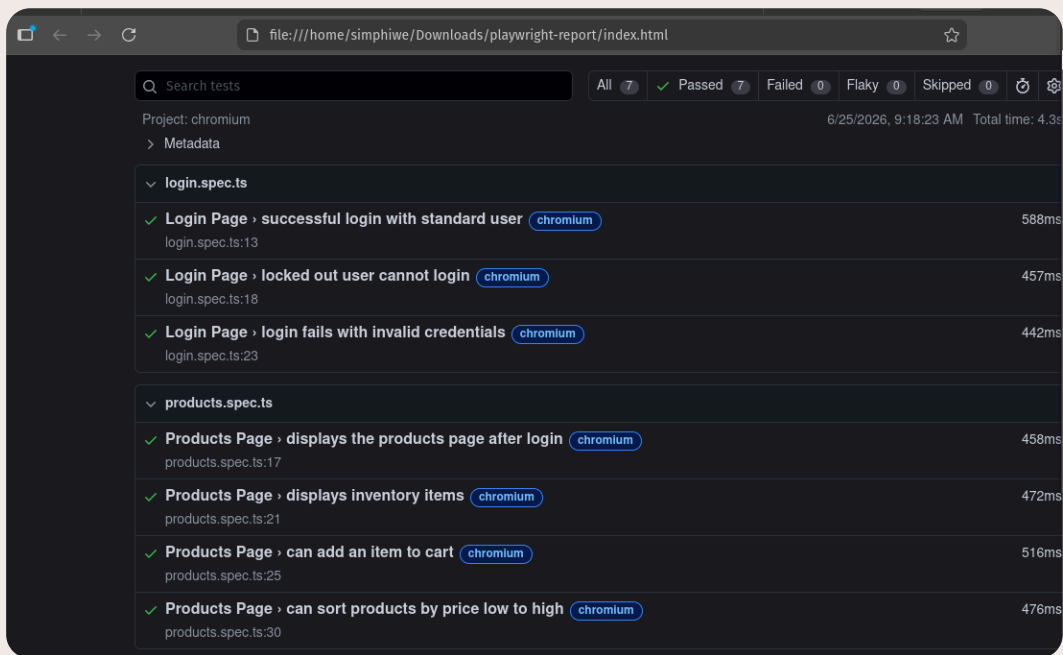


BUILD A PLAYWRIGHT E2E TEST PORTFOLIO

By simphiwe mzazi | June 2026



Project Vision: Automating Real-World QA

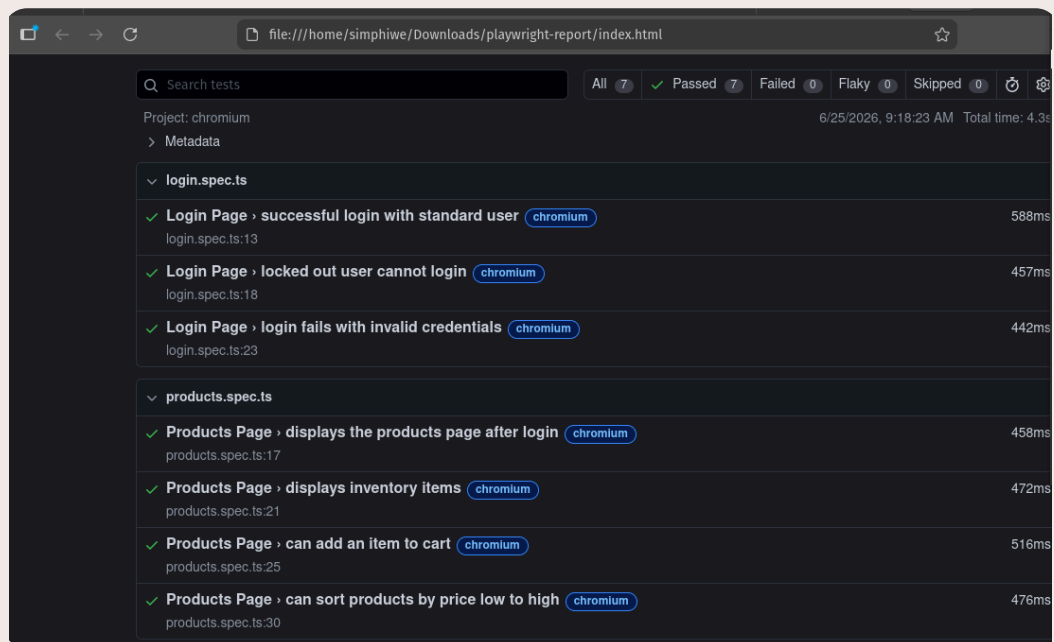
Why this project matters for a QA career

In this project, I'm building tests for a e-commerce web using a Playwright test suite so that I can automatically tests a real e-commerce web application

Deploying CI/CD and Going Live on GitHub

Setting up automated pipelines

In this step, I'm setting up GitHub Actions workflow for automated test runs so that I can Push to GitHub and confirm CI passes with a downloadable HTML report.



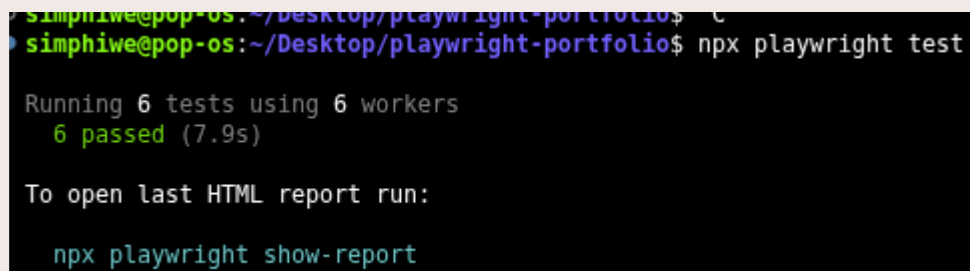
How CI keeps the test suite reliable

Every time I push, GitHub Actions automatically lets you define automated workflows that run on every push or pull request.

Setting Up the Playwright Project

Initializing the test framework

In this step, I'm setting up Playwright so that I can configure the project and push it to GitHub repository

A terminal window screenshot showing the execution of Playwright tests. The prompt is 'simphiwe@pop-os: ~/Desktop/playwright-portfolio\$'. The command 'npx playwright test' is entered. The output shows 'Running 6 tests using 6 workers', '6 passed (7.9s)', and instructions to open the HTML report by running 'npx playwright show-report'.

```
simphiwe@pop-os: ~/Desktop/playwright-portfolio$ npx playwright test

Running 6 tests using 6 workers
  6 passed (7.9s)

To open last HTML report run:
  npx playwright show-report
```

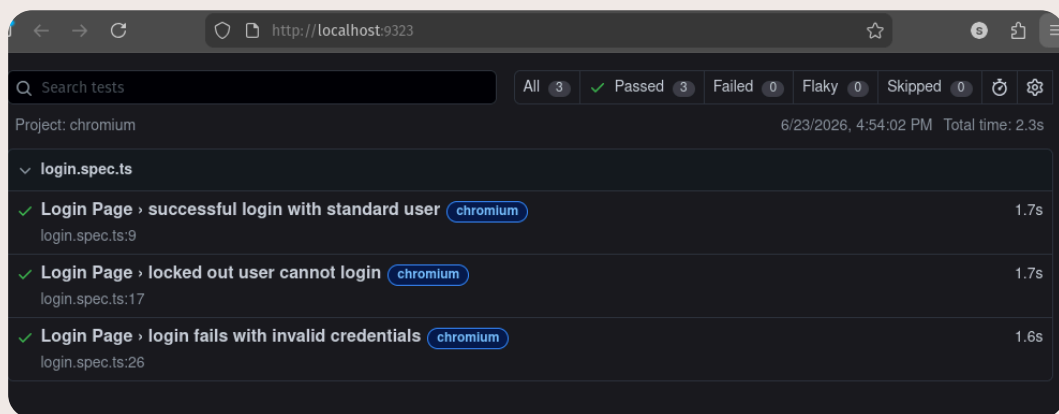
Published repository

<https://github.com/simzacpt025-crypto/playwright-portfolio>

Writing Login Tests: Positive and Negative Scenarios

Planning the first test cases

In this step, I'm writing my first login tests so that I can cover positive and negative testing



Why both positive and negative tests matter

My three tests cover successful login with standard user , locked out user cannot login , login fails with invalid credentials so both positive and negative cases matters because it ensures that an application behaves correctly when users provide both valid and invalid inputs

Refactoring with the Page Object Model

Applying professional architecture patterns

In this step, I'm setting up a LoginPage so that I can refactor login tests and verify all tests still pass after the refactor

```
simphiwe@pop-os:~/Desktop/playwright-portfolio$ npx playwright test
Running 3 tests using 3 workers
 3 passed (2.7s)

To open last HTML report run:
  npx playwright show-report
```

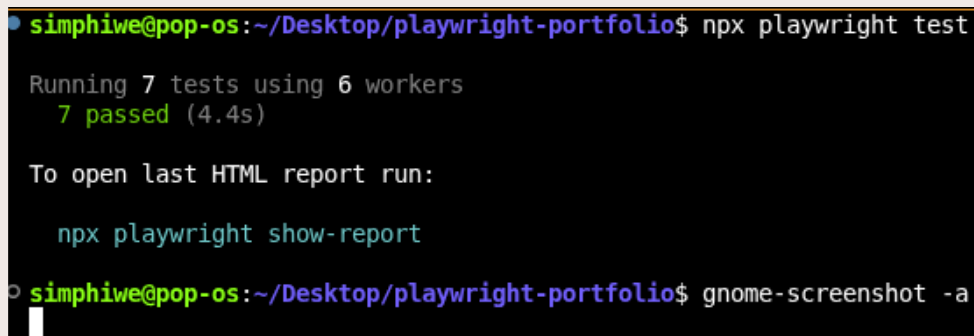
The value of POM for test maintenance

I learned that POM helps because you update one line in LoginPage.ts and every test that uses it is fixed automatically.

Testing Product Interactions

Expanding test coverage to product features

In this step, I'm testing 4 product tests covering display, cart, and sorting so that my portfolio demonstrates ProductPage class with locators and an add-to-cart method

A terminal window with a dark background and light-colored text. The prompt is 'simphiwe@pop-os:~/Desktop/playwright-portfolio\$'. The command 'npx playwright test' has been executed. The output shows 'Running 7 tests using 6 workers' and '7 passed (4.4s)'. Below this, it says 'To open last HTML report run:' followed by the command 'npx playwright show-report'. The prompt is now 'simphiwe@pop-os:~/Desktop/playwright-portfolio\$' and the command 'gnome-screenshot -a' has been entered, with a cursor at the end.

```
simphiwe@pop-os:~/Desktop/playwright-portfolio$ npx playwright test

Running 7 tests using 6 workers
  7 passed (4.4s)

To open last HTML report run:

  npx playwright show-report

simphiwe@pop-os:~/Desktop/playwright-portfolio$ gnome-screenshot -a
```

Using test.beforeEach for clean test setup

I used test.beforeEach because it hooks logs in with standard_user before every test. This demonstrates a professional pattern: isolating test setup from test assertions.

Test 1 verifies the page title reads "Products" after login.

Test 2 confirms at least one inventory item is visible on the page.

Test 3 adds the Sauce Labs Backpack to the cart and asserts the cart badge updates to "1".

Test 4 selects "Price (low to high)" from the sort dropdown and checks the cheapest item shows

Reflections and Key Takeaways

Tools and concepts mastered

The key tools I used include Playwright , Json , VS Code , HTML ans swagsauce website the Key concepts I learnt include Refractor , Page Object Model and GitHub Actions CI pipeline

Time and challenges

This project took me approximately 2days The most challenging part was refactoring getting bugs in the process and understanding how to move on to the next requirements for a successful testing

Looking ahead

I did this project today to learn how to Tackle Software Testing and Test Specification Another skill I want to learn is Penetration Tester